# REAL-TIME KUBERNETES MICROSERVICES

## Production DevOps Project  Food Delivery Platform

A hands-on guide for React, FastAPI microservices, Docker, Kubernetes, Helm, Terraform, Azure AKS, CI/CD, security and observability.

## 1. Project Overview

React frontend, API Gateway, User, Restaurant, Order, Payment and Notification services. PostgreSQL provides durable data; Redis provides caching and temporary order state.

## 2. Runtime Architecture

User → Ingress → Frontend/API Gateway → microservices → PostgreSQL/Redis. Kubernetes Services provide internal discovery. Probes, replicas, rolling updates, HPA, PDB and NetworkPolicy support production-style operations.

## 3. Local Docker

Run ‘docker compose build‘, ‘docker compose up -d‘, ‘docker compose ps‘, then open port 8080. Use ‘docker compose logs -f‘ for troubleshooting and ‘docker compose down‘ for cleanup.

## 4. Docker Security

Application images use slim/alpine bases, non-root execution where practical, dropped capabilities, health checks and .dockerignore files. Scan images with Trivy before pushing.

## 5. Kubernetes

Apply namespace, ConfigMap, Secret, RBAC, database/cache, service deployments, HPA, Ingress and NetworkPolicy. Each application has startup, readiness and liveness probes plus resource requests/limits.

## 6. Helm

Use ‘helm lint‘, ‘helm template‘, ‘helm upgrade --install‘, ‘helm rollback‘ and ‘helm uninstall‘. Values control registry, image tag, replicas, database and ingress settings.

## 7. Terraform + AKS

Terraform provisions resource group, VNet/subnet, AKS, ACR, Log Analytics and the AKS kubelet AcrPull assignment. Review region, VM size, scaling, networking and security before production.

## 8. CI/CD

Git push → validation/tests → Docker build → Trivy scan → registry push → AKS image update → rollout status → smoke checks. Use workload identity/OIDC or approved federated credentials rather than long-lived secrets.

## 9. Security

Never commit real credentials. Use an external secret manager such as Azure Key Vault with CSI or another approved platform. Apply least-privilege RBAC and NetworkPolicy. Prefer immutable image tags or digests in production.

## 10. Monitoring & Logging

Prometheus/Grafana should track CPU, memory, restarts, API rate/latency, HPA and database health. Use kubectl logs, logs --previous and describe pod during incidents. Centralized logging should aggregate structured application logs.

## 11. Troubleshooting

CrashLoopBackOff: inspect current/previous logs and environment. ImagePullBackOff: inspect pod events and registry permissions. Pending: inspect scheduling/PVC events. OOMKilled: inspect metrics and tune limits. Probe failure: test endpoint and startup window. Ingress/service/DNS failures: inspect endpoints, controller logs and NetworkPolicy.

## 12. Disaster Recovery

Take PostgreSQL logical backups, store them in durable encrypted storage, test restores, retain immutable container images, and keep deployment rollback procedures documented. Define RPO/RTO with stakeholders.

## 13. AWS/GCP Equivalents

For AWS, use EKS + ECR and cloud-native IAM/load balancing/storage. For GCP, use GKE + Artifact Registry and Workload Identity. Kubernetes application manifests remain largely portable.

## 14. Interview Preparation

Be ready to explain API Gateway, service discovery, probes, HPA/PDB, rolling updates, NetworkPolicy, immutable tags, registry security, CI/CD gates, database resilience and observability.

# Useful Commands

```
docker compose build
docker compose up -d
docker compose ps
docker compose logs -f

terraform init
terraform validate
terraform plan
terraform apply

kubectl -n food-delivery get pods,svc,ingress,hpa
kubectl -n food-delivery logs -f deployment/order-service
kubectl -n food-delivery describe pod <pod>
kubectl -n food-delivery rollout status deployment/order-service
kubectl -n food-delivery rollout undo deployment/order-service

helm lint helm/food-delivery
helm template food-delivery helm/food-delivery
helm upgrade --install food-delivery helm/food-delivery -n food-delivery --create-namespace

trivy image --severity HIGH,CRITICAL --ignore-unfixed <image>
```